Course 50.050/50.550
Advanced Algorithms

Week 5 – Lecture L05.02

Outline of Lecture

- Implementing graph algorithms
- Underlying graphs, connectedness, connected components
- Computing bridges using depth-first search
- Example: Implementing Fleury's algorithm

How to think about implementing graph algorithms

1. Be very clear about the graph-theoretic **definitions** you are using.
   - e.g. an Eulerian path is an alternating sequence of vertices and edges.
   - Be consistent in your terminology/notation!
2. Be very clear about the problem you wish to solve, including any **assumptions** made about the given graph.
   - e.g. Given a **connected** **undirected** graph G that has an Eulerian path, construct an Eulerian path in G.
3. Understand the **ideas** that can be used to solve the problem.
   - Fleury's algorithm: Always traverse **non-bridges** whenever possible.
4. Choose a suitable **graph representation** for your identified ideas.
   - Incidence list/matrix? Adjacency list/matrix? Mix?
5. Write a (very informal) **initial pseudocode** to combine your ideas.
   - Understand the rough structure of your algorithm (e.g. need while loop?)
6. Introduce algorithm **subroutines** for your different ideas.
   - Always keep your algorithm as easily readable/understandable as possible. Be generous with comments in your algorithm.
7. Fill in details for your **finalized pseudocode**.
   - Be careful of "exceptional" cases.

Most difficult step? Step 3, **ideas** for problem-solving.

Divide-and-conquer for graph-based problems

General strategy: Define **subproblems** of your original problem.

- Can you first consider a "simpler" graph?
  - If your graph is non-simple in general, you could try solving the problem first in the special case of simple graphs.
    - Ask yourself: What happens if you remove loops and multiple edges from your input graph? Would the answer to your problem remain invariant? Or would the answer only change just slightly?
    - You would need the idea of "underlying simple graph".
  - If your graph is directed, would solving the analogous problem in the undirected case help?
    - You would need the idea of "underlying undirected graph".
- Can you decompose your graph into multiple "simpler" subgraphs?
  - If your graph is disconnected in general, you could try solving the problem first on a connected subgraph.
    - You would need the idea of "connected components".
- Can you solve your problem partially, and modify your graph into a "simpler graph"?
  - After "processing" a vertex, can you delete the vertex?
  - After "processing" an edge, can you delete the edge?
    - Do you want to do anything to any isolated vertices that are introduced after deleting edges?

Graph algorithm convention for 50.050

In many programming languages, graphs are not implemented data structures.

- You would have to implement your own graph data structure.
- There are many ways to implement your own graph data structure.

A graph is formally defined as a 3-tuple $G = (V, E, \phi)$. Hence, for 50.050, we shall assume that for algorithms, graphs are data structures, where every graph G has three attributes $G.V$, $G.E$, $G.phi$.

- $G.V$ is an array (e.g. $[v_1, \ldots, v_n]$) consisting of all vertices in G.
  - Vertices are objects. For convenience, we assume that we can iterate over the vertices by "**for** each vertex v in $G.V$".
- $G.E$ is an array (e.g. $[e_1, \ldots, e_m]$) consisting of all edges in G.
  - Similarly, edges are objects, and we assume that we can iterate over the edges by "**for** each edge e in $G.E$".
  - Edges could be 2-subsets of $G.V$.
- $G.phi$ represents the incidence function ϕ of G, where for each edge e in $G.E$, $G.phi[e]$ represents $\phi(e)$.
  - If G is directed, then $G.phi[e]$ is an array $[u, v]$ that represents the 2-tuple $\phi(e) = (u, v)$.
  - If G is undirected, then $G.phi[e]$ is a set that represents the set $\phi(e)$, which could have 1 or 2 elements. We assume that sets are implemented, and that $\{u, v\}$ is a set with elements u, v.

Graph algorithm convention for 50.050 (continued)

We shall write "$G = (G.V, G.E, G.phi)$ is a directed graph" or write "$G = (G.V, G.E, G.phi)$ is an undirected graph" to mean that we use the graph data structure described in the previous slide.

- We may assume that G has an additional attribute $G.Inc$, representing the incidence list, where for each vertex $v \in G.V$, $G.Inc[v]$ is a linked list whose elements are the edges incident to v.
- We may assume that G has an additional attribute $G.Adj$, representing the adjacency list, where for each vertex $v \in G.V$, $G.Adj[v]$ is a linked list whose elements are either neighbors of v (when G is undirected), or out-neighbors of v (when G is directed).
- **Note:** For all $v \in G.V$, $G.Inc[v]$ and $G.Adj[v]$ are linked lists.
  - **Recall:** For any linked list L, the head of L is accessed by L.head. Each element x of L has two attributes x.prev and x.next.
  - We have access to operations List-Search, List-Delete, List-Insert; see [CLRS] (50.004 Algorithms course textbook).

Underlying simple graphs

Definition: Let $G = (V, E, \phi)$ be an undirected or directed graph. The underlying simple graph of G is a simple graph $G' = (V', E')$ such that $V' = V$, and E' is the set $\{\phi(e) | e \in E, e \text{ is not a loop}\}$.

function UNDERLYINGSIMPLEGRAPH(G)
Require: $(G.V, G.E, G.phi)$ is a graph.
Initialize new graph G'
$G'.V \leftarrow G.V$
Initialize empty set E'
for each edge e in $G.E$ **do**
  if not ISLOOP(G, e) **then**
    Insert $G.phi(e)$ into E'
$G'.E \leftarrow E'$
$G'.phi \leftarrow$ identity map $x \mapsto x$
return G'

function ISDIRECTED(G)
Require: $(G.V, G.E, G.phi)$ is a graph.
if $G.E$ is not empty **then**
  Randomly select an edge e in $G.E$
  if $G.phi[e]$ is not a set **then**
    return `True`
return `False`

function ISLOOP(G, e)
Require: $(G.V, G.E, G.phi)$ is a graph.
Require: e is an edge in $G.E$.
if not ISDIRECTED(G) **then**
  if $|G.phi[e]| = 1$ **then**
    return `True`
  else
    return `False`
else
  $[u, v] \leftarrow G.phi[e]$
  if $u = v$ **then**
    return `True`
  else
    return `False`

Underlying undirected graphs

Definition: Let $G = (V, E, \phi)$ be a **directed** graph. The underlying undirected graph of G is an undirected graph $G' = (V', E', \phi')$ such that $V' = V$, $E' = E$, and ϕ' is a function on E' given by:

- If $\phi(e) = (v, v)$ for some $v \in V'$, then $\phi'(e) = \{v\}$.
- If $\phi(e) = (u, v)$ for some distinct $u, v \in V'$, then $\phi'(e) = \{u, v\}$.

function UNDERLYINGUNDIRECTEDGRAPH(G)

Require: $G = (G.V, G.E, G.phi)$ is a directed graph.

1: **for** each edge e in $G.E$ **do**
2: $[u, v] \leftarrow G.phi[e]$
3: $G.phi[e] \leftarrow \{u, v\}$

Note: Here, we assume that sets are implemented so that repeated elements are "ignored", i.e. if u, v are the same object, then $\{u, v\}$ and $\{u\}$ are equal as sets.

Definition: Let $G = (V, E, \phi)$ be a **directed** graph. The underlying simple undirected graph of G is the underlying simple graph of the underlying undirected graph of G.

function UNDERLYINGSIMPLEUNDIRECTEDGRAPH(G)

Require: $G = (G.V, G.E, G.phi)$ is a directed graph.

1: UNDERLYINGUNDIRECTEDGRAPH(G)
2: **return** UNDERLYINGSIMPLEGRAPH(G)

Edge deletion

Definition: Let $G = (V, E, \phi)$ be a graph, and let $e \in E$ be any edge. The deletion of e from G, denoted by $G \backslash e$ or $G - e$, is the graph $G' = (V', E', \phi')$ where $V' = V$, $E' = E \backslash \{e\}$, and where the incidence function ϕ' is the restriction of ϕ to domain E'.

function DELETEEDGE(G, e)

Require: $G = (G.V, G.E, G.phi)$ is a graph.

Require: $G.Inc$ is the incidence list of G.

Require: e is an edge in $G.E$.

1: **for** each vertex v in $G.V$ **do**

2: LIST-DELETE($G.Inc[v], e$)

3: Delete entry e from array $G.E$

Note: The deletion of an entry from an array is "a standard operation" and depends on the programming language.

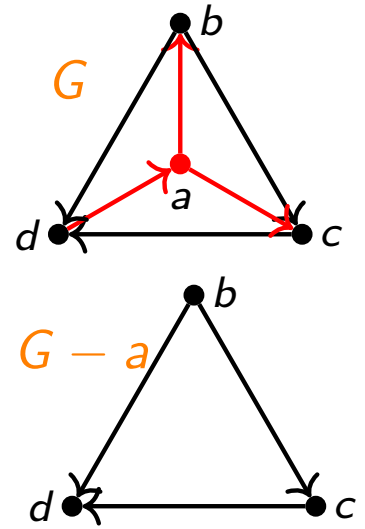
- The operation to modify G into $G \backslash e$ is also called edge deletion, or deleting e from G.
- **Technicality:** Although we did not delete the value of $\phi(e)$ from the machine's memory, this value is no longer accessible from any properly implemented code, since for any graph G, we only access $G.phi[e']$ for those edges e' in $G.E$, and $G.E$ no longer contains the edge e after the deletion of e.

Vertex deletion

Definition: Let $G = (V, E, \phi)$ be a graph, and let $v \in V$. The deletion of v from G, denoted by $G - v$ (or $G \backslash v$), is the graph $G' = (V', E', \phi')$ where $V' = V \backslash \{v\}$, $E' = E \backslash \{\text{all edges incident to } v\}$, and the incidence function ϕ' is the restriction of ϕ to domain E'.

function DELETEVERTEX(G, v)
  Require: ($G.V, G.E, G.phi$) is a graph.
  Require: $G.Inc$ is the incidence list of G.
  Require: v is a vertex in $G.V$.
1: Initialize empty array *toDelete*
2: **for** e in $G.E$ **do**
3: **if** $G.phi[e]$ contains v **then**
4: Append *toDelete* with e
5: **for** e in *toDelete* **do**
6: **for** each vertex u in $G.V$ **do**
7: LIST-DELETE($G.Inc[u], e$)
8: Delete entry e from array $G.E$
9: Delete entry v from array $G.V$

Example:



Connectedness and connected components

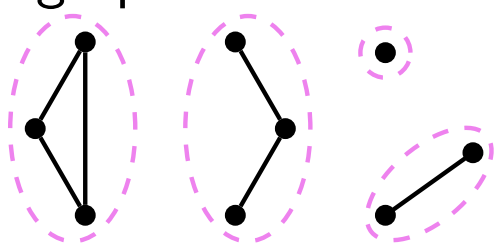
Definition: Let $G = (V, E, \phi)$ be an **undirected** graph. If u and v are connected for all possible vertices u, v in V, then we say that G is connected, otherwise we say that G is disconnected.

- A connected component of G is a connected vertex-induced subgraph $G[U]$ of G for some subset $U \subseteq V$, such that $G[U']$ is disconnected for all $U \subsetneq U' \subseteq V$ (if any).
  - Recall: $G[U]$ is the unique subgraph of G with the largest possible edge set, whose vertex set is exactly U.

Intuition: Given a graph G with $V(G) \neq \emptyset$, we can partition $V(G)$ into disjoint non-empty subsets $U_1, \ldots, U_\ell$ (for some $\ell \in \mathbb{Z}^+$), such that the vertex-induced subgraphs $G[U_1], \ldots, G[U_\ell]$ contains all edges in $E(G)$.

- This number ℓ is called the number of connected components in G, and it is a graph invariant.

Example: An undirected graph with 4 connected components.



Bridges in a graph

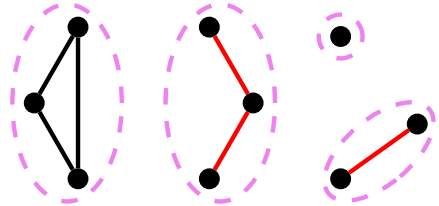
In L05.01, we defined bridges in connected undirected graphs.

- Now, we define bridges in arbitrary undirected graphs.

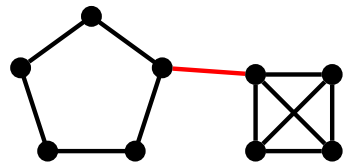
Definition: Let $G = (V, E, \phi)$ be an **undirected** graph, and let $e \in E$ be an edge. If the number of connected components in the deletion $G \backslash e$ is strictly larger than the number of connected components in G, then e is called a bridge (or cut edge) of G.

- **Intuition:** The deletion of a bridge will increase the number of connected components in the graph.

Example: An undirected graph with 3 bridges (depicted as red edges).



Example: An undirected graph with 1 bridge (depicted as a red edge).



Theorem: Let $G = (V, E, \phi)$ be an **undirected** graph. An edge $e \in E$ is a bridge of G if and only if e is not an edge of a cycle in G.

Note: Bridges in a graph can be computed using depth-first search!

Recap: Depth-first search (DFS)

Set-up for DFS from 50.004: Let $G = (V, E, \phi)$ be a graph.

- Every vertex $x \in V$ has the following **four attributes**:
  - $x.color \in \{\text{'WHITE'}, \text{'GRAY'}, \text{'BLACK'}\}$, a "color" attribute.
    - All vertices are initially 'WHITE', representing "unvisited" vertices.
    - When a vertex is visited for the first time (i.e. "discovered"), the vertex is colored 'GRAY'.
    - When a vertex is visited for the last time (i.e. "finished"), the vertex is colored 'BLACK'.
  - $x.d \in \mathbb{Z}^+$, a "discovery time" attribute representing the time that x was first visited.
  - $x.f \in \mathbb{Z}^+$, a "finishing time" attribute representing the time that x was last visited.
  - $x.\pi \in V$, an attribute representing the predecessor of x.
- **All vertices are visited** in a DFS traversal of G, whether or not G is connected.
- Whether or not G is directed, we can construct a DFS forest of G (also called a DFS tree or a predecessor subgraph of G).
  - **Note:** "DFS forest" is the most accurate terminology. This graph is indeed a forest, and is in general not a tree. Also, this DFS forest is a directed graph, so it is not a subgraph of G when G is undirected. We will study trees and forests in more detail in Week 6.

Recap: Pseudocode for Depth-first search (DFS)

function DFS(G)
Require: $G = (G.V, G.E, G.phi)$ is a graph.
Require: $G.Inc$ is the incidence list of G.
1: **for** each vertex u in $G.V$ **do**
2: $u.color \leftarrow$ 'WHITE'
3: $u.\pi \leftarrow$ NIL
4: $time \leftarrow 0$
5: **for** each vertex u in $G.V$ **do**
6: **if** $u.color =$ 'WHITE' **then**
7: DFS-Visit(G, u)

function otherEndpoint(G, e, u)
Require: $G = (G.V, G.E, G.phi)$ is a graph.
Require: e is an edge in $G.E$.
Require: u is a vertex in $G.V$ incident to e.
1: **if** isLoop(G, e) **then**
2: **return** u
3: **else**
4: **return** vertex in $G.phi[e]$ not u

function DFS-Visit(G, u)
Require: $G = (G.V, G.E, G.phi)$ is a graph.
Require: $G.Inc$ is the incidence list of G.
Require: u is a vertex in $G.V$.
1: $time \leftarrow time + 1$
2: $u.d \leftarrow time$
3: $u.color \leftarrow$ 'GRAY'
4: $e \leftarrow G.Inc[u]$.head
5: **while** $e \neq$ NIL **do**
6: $v \leftarrow$ otherEndpoint(e, u)
7: **if** $v.color =$ 'WHITE' **then**
8: $v.\pi \leftarrow u$
9: DFS-Visit(G, v)
10: $e \leftarrow e$.next
11: $u.color \leftarrow$ 'BLACK'
12: $time \leftarrow time + 1$
13: $u.f \leftarrow time$

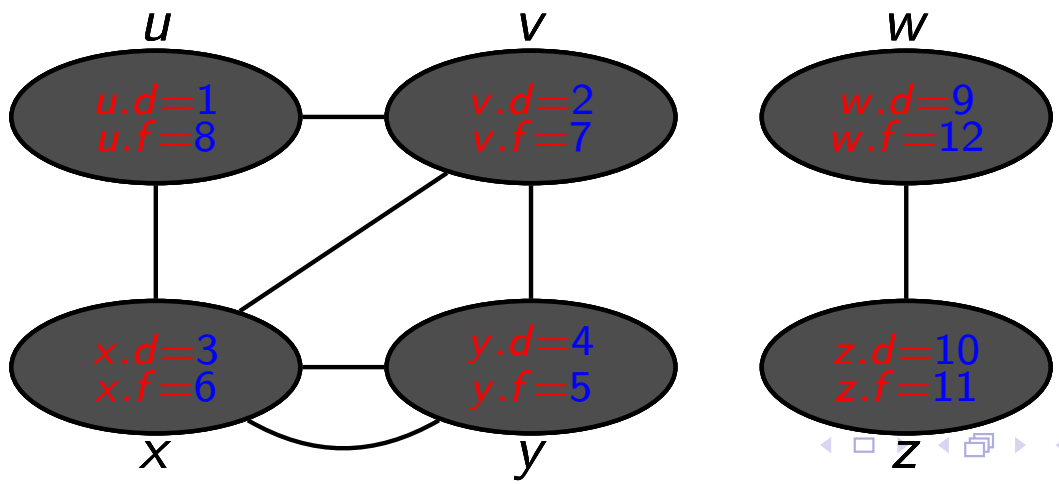
Differences from [CLRS] pseudocode:

- [CLRS] uses adjacency list, which is ambiguous for non-simple graphs.
- [CLRS] accesses elements in linked list via for-loop, which is technically wrong.

Recap: Intuition for DFS for undirected graphs

- After a vertex u has been discovered, check if u is incident to any edge with an undiscovered endpoint.
  - If u is incident to an edge with an undiscovered endpoint v, then traverse to v, and record the discovery time for v.
    - Check incident edges based on the order given by $G.Inc[u]$.
  - If not, then record the finishing time for u, and back-track to the predecessor of u.
- Continue process until all vertices reachable from the original "source vertex" have been discovered.
  - If any undiscovered vertices remain, pick any one of them as the "new source vertex", and repeat this process iteratively, until all vertices have been discovered.

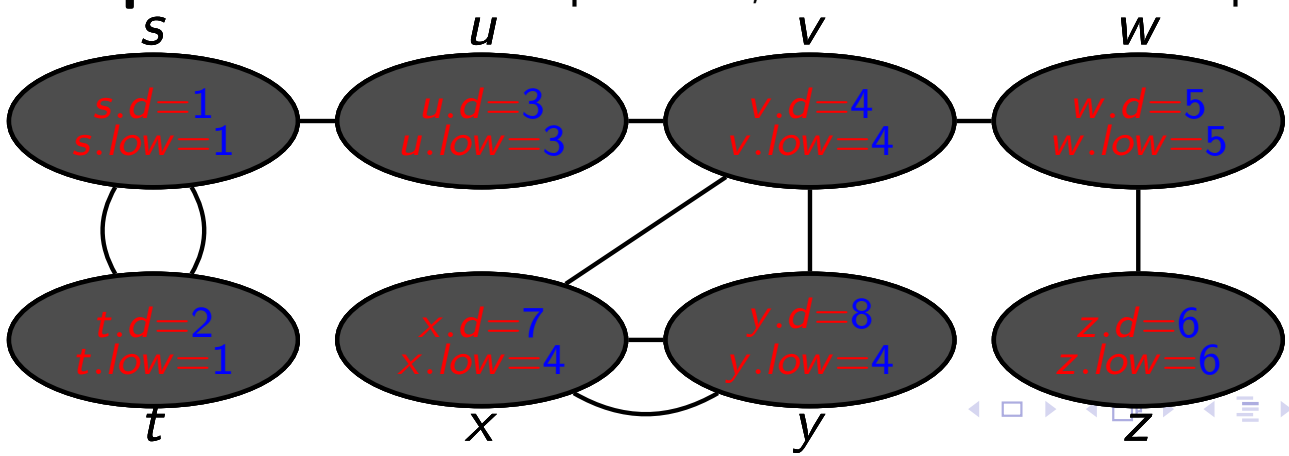
DFS Example: Within each depth level, traverse vertices in alphabetical order.



Modified DFS with lowest discovery time

- Modify attribute $u.\pi$ so that $u.\pi \in G.E$ is the edge immediately preceding u in the DFS traversal.
- For every vertex $u \in V$, introduce a new attribute $u.low$, which represents the "**lowest discovery time**".
  - If u is incident to an edge with an undiscovered endpoint v, then traverse to v, and record the discovery time for v as usual.
  - If not, then find the smallest possible value among $v.d$ and $v.low$ as v ranges through u and all endpoints of **non-predecessor** edges incident to u, record $u.low$ with this smallest possible value, then back-track to $u.\pi$.
- Continue process until all vertices reachable from the original "source vertex" has been discovered. Iterate process on remaining undiscovered vertices, until all vertices have been discovered.

DFS Example: Within each depth level, traverse vertices in alphabetical order.



Pseudocode for Modified DFS

function DFS-LOW(G)
Require: $G = (G.V, G.E, G.phi)$ is a graph.
Require: $G.Inc$ is the incidence list of G.
1: **for** each vertex u in $G.V$ **do**
2: $u.color \leftarrow$ 'WHITE'
3: $u.\pi \leftarrow$ NIL
4: $time \leftarrow 0$
5: **for** each vertex u in $G.V$ **do**
6: **if** $u.color =$ 'WHITE' **then**
7: DFS-VISIT-LOW(G, u)

function DFS-VISIT-LOW(G, u)
1: $time \leftarrow time + 1$
2: $u.d \leftarrow time$
3: $u.color \leftarrow$ 'GRAY'
4: $e \leftarrow G.Inc[u]$.head
5: **while** $e \neq$ NIL **do**
6: $v \leftarrow$ OTHERENDPOINT(e, u)
7: **if** $v.color =$ 'WHITE' **then**
8: $v.\pi \leftarrow e$
9: DFS-VISIT-LOW(G, v)
10: $e \leftarrow e$.next
11: $e \leftarrow G.Inc[u]$.head
12: $\ell \leftarrow u.d$
13: **while** $e \neq$ NIL **do**
14: **if** $e \neq u.\pi$ **then**
15: $v \leftarrow$ OTHERENDPOINT(e, u)
16: **if** $u.d < v.d$ **then**
17: $\ell \leftarrow \min\{\ell, v.low\}$
18: **else**
19: $\ell \leftarrow \min\{\ell, v.d\}$
20: $e \leftarrow e$.next
21: $u.color \leftarrow$ 'BLACK'
22: $u.low \leftarrow \ell$

Identifying bridges in terms of lowest discovery time

Theorem: Let $G = (G.V, G.E, G.phi)$ be an **undirected** graph, and suppose we have already run DFS-LOW(G). Let $e \in G.E$ be a non-loop, and suppose that $G.phi[e] = \{u, v\}$ for some distinct vertices $u, v \in G.V$ such that $u.d < v.d$. Then e is a bridge if and only if $u.d < v.low$.

Intuition:

- A loop is never a bridge.
- Given a non-loop e with endpoints u, v, the assumption that $u.d < v.d$ means that u was discovered before v in the DFS traversal.
- If $v.low \leq u.d$, then it means that some vertex w discovered no earlier than v is adjacent to a vertex discovered no later than u.
  - i.e. a "subsequent" vertex w is adjacent to a vertex that is either u or an earlier vertex.
  - Then there would be a cycle from w to u with edge e. This forces e to not be a bridge.

Useful auxiliary graph algorithms

function isBridge(G, e)
Require: $G = (G.V, G.E, G.phi)$ is an undirected graph.
Require: e is an edge in $G.E$.

```
1: DFS-low(G)
2: {u, v} ← G.phi[e]
3: if u.d < v.d then
4:     if u.d < v.low then
5:         return True
6:     return False
7: else
8:     if u.d > v.low then
9:         return True
10:    return False
```

function suitableStart(G)
Require: $G = (G.V, G.E, G.phi)$ is an undirected graph.
Require: $G.Inc$ is the incidence list of G.

```
1: for each vertex v in G.V do
2:     count ← 0
3:     e ← G.Inc[v]
4:     while e ≠ NIL do
5:         count ← count + 1
6:         e ← e.next
7:     if count is odd then
8:         return v
9: return randomly chosen vertex in G
```

Recap: Idea of Fluery's algorithm

An algorithm that takes as its input a connected undirected graph G that has an Eulerian path, and returns as its output an Eulerian path.

- **Note:** Fleury's algorithm iteratively modifies G. (G is "destroyed".)
- **Recall:** An Eulerian path in a connected undirected graph G is a trail in G that contains all edges in $E(G)$.
  - A trail is an alternating sequence of vertices and edges.

Idea of Fluery's algorithm: For constructing an Eulerian path **p**.

1: $x \leftarrow$ a suitable vertex v_0 of G as starting vertex for an Eulerian path in G
2: **p** $\leftarrow [x]$
3: **while** G has at least one edge **do**
4: **if** x is incident to at least one non-bridge **then**
5: $e \leftarrow$ any randomly selected non-bridge incident to x
6: $x \leftarrow$ the other endpoint of e
7: **else**
8: $e \leftarrow$ the bridge incident to x *(Note:* $\deg(x)$ *must equal* 1 *in this else-case.)*
9: $x \leftarrow$ the other endpoint of e
10: Append **p** by e and x
11: Delete from G the edge e and any (new) isolated vertices (after deleting e)
12: **return p**

Implementation of Fluery's algorithm

function CONSTRUCTEULERIANPATH(G)
  Require: $G = (G.V, G.E, G.phi)$ is a connected undirected graph.
  Require: $G.Inc$ is the incidence list of G.
1: $x \leftarrow$ SUITABLESTART(G)
2: **p** $\leftarrow [x]$
3: **while** $|G.E| \geq 1$ **do**
4: $e \leftarrow G.Inc[x]$.head
5: $y \leftarrow$ OTHERENDPOINT(G, e, x)
6: **while** $e \neq$ `NIL` **do**
7: **if** not ISBRIDGE(G, e) **then**
8: $y \leftarrow$ OTHERENDPOINT(G, e, x)
9: **break**
10: $e \leftarrow e$.next
11: Append **p** by e and y
12: DELETEEDGE(G, e)
13: **if** $G.Inc[x]$.head $=$ `NIL` **then**
14: DELETEVERTEX(G, x)
15: $x \leftarrow y$
16: **return p**

Summary

- Implementing graph algorithms
- Underlying graphs, connectedness, connected components
- Computing bridges using depth-first search
- Example: Implementing Fleury's algorithm